

研究进展汇报

Asynchronous Delayed SNN

从 Level 0 机制诊断到 K=5 时间路由

汇报讲稿 · 结论、证据链与失败机制

一句话结论 Oracle 与显式 timing scaffold 证明共享神经元具有理论时间复用能力；但当前 task-driven 训练不能可靠地从头发现路由，突然撤掉 scaffold 也不能稳定保住内部 schedule。问题更像是 credit assignment、可辨识性和参数化几何共同造成，而不是 simulator 完全不支持 delay。

证据截止：2026-07-17 06:03 (含最新 W1 machine decision)

1. 开场：我现在愿意支持什么结论

推荐开场白 我的结论不是“delayed SNN 已经学会了 time multiplexing”，也不是“这个想法完全不成立”。更准确的说法是：系统存在可实现的时间复用解，但当前训练方法不能自主、稳定地找到这个解。

三个层次必须分开

1. 表达能力：固定 oracle schedule 能否让同一组 hidden neurons 在不同时间窗口处理不同 query？当前答案是可以。
2. 可训练能力：仅靠 task loss, delay 能否从普通初始化自动形成 query-to-window routing？当前答案是否定的。
3. 资源优势：即使 hidden area 有复用，dense MAC、events、decoder 和 delay-buffer 是否同时下降？当前没有证据，部分成本反而增加。

因此，“理论复用能力”是架构层面的可表达性结论；“训练不出来”是优化与任务可辨识性的结论；两者并不矛盾。

措辞边界 建议全程使用 temporal reuse / temporal routing feasibility，而不是把所有 positive oracle result 都称为 learned temporal multiplexing。

2. 整条实验链回答了什么

阶段	核心问题	结果	对主结论的作用
Level 0A	delay 参数本身能否移动?	强监督 + 合适 LR 可恢复	排除硬实现错误
Level 0B	buffer/LIF 是否传递双向 timing credit?	严格 gate 失败	定位 loss 几何问题
Level 0C/0D	连续 timing auxiliary 能否跨 hard output?	soft centroid bridge 通过	证明可优化但需 scaffold
Level 1A/1B	加入 XOR 与多 delay 坐标后是否仍可学?	scaffold 可学, task-only 失败	训练与表达能力分离
Withdrawal	撤掉 teacher 后能否恢复 schedule?	功能保留, 局部恢复失败	暴露边界梯度与不可辨识
K=5 mixed-op	共享 hidden 是否能跨五窗口复用?	oracle 成功, WAD 失败	最直接的理论复用证据
V3 / W0 / W1	修正目标并撤 scaffold 后怎样?	显式 centroid 成功; 退火保留, 突然撤除不稳	学习仍依赖课程与先验

3. Level 0: 先判断是不是底层实现根本学不动

3.1 Level 0A: 不是 delay 参数完全不可优化，而是旧 recipe 太弱

旧设置 $lr=.001$ 、200 steps，从 $delay=.9536$ 去 $target=5$ ，只到 1.1389，最终误差 3.8611。直接监督下， $lr=.05$ 可以恢复全部 15/15 interior target/init 组合，最大误差 .0781。

- 排除项：production sigmoid delay 并不存在一个硬性的 `get_delays()` 数值错误。
- 保留项：边界目标仍有 sigmoid 饱和几何；旧 LR/预算不足以支持长距离路由。

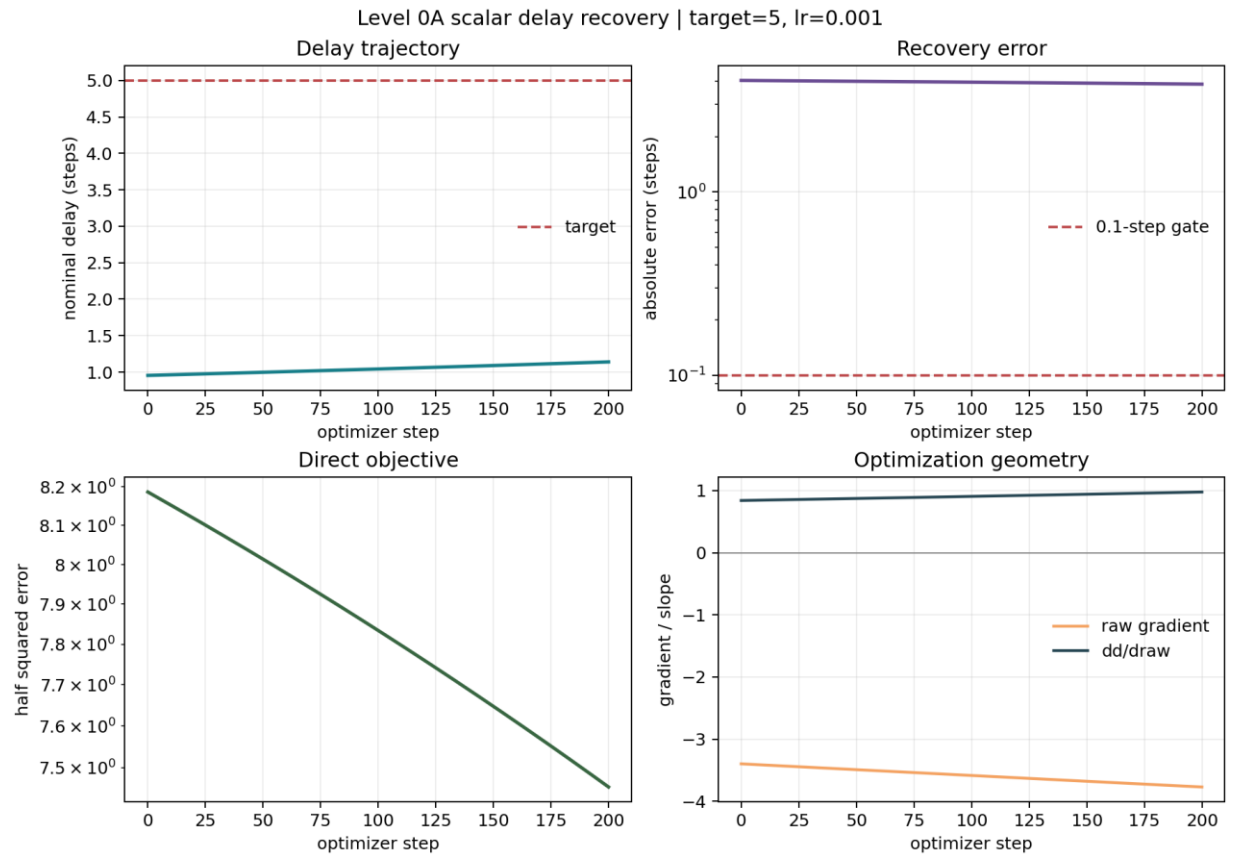
3.2 Level 0B: 真正的问题开始出现在 temporal credit

buffer-current centroid 能恢复 15/15，说明 circular buffer 并没有阻断梯度。但 causal filtered trace 只有 5/15，hard-spike centroid 只有 3/15，hard-LIF filtered trace 只有 10/15。

关键诊断不是“有没有梯度”，而是梯度方向是否正确：filtered objective 在 13 个失配组合中有 5 个初始方向错误；hard-spike centroid 有 8 个零梯度。

这一页要强调 Level 0 把“代码完全坏了”和“credit 不好用”分开了。后续失败不能简单归因于 buffer bug，也不能因为 gradient norm 非零就认为训练信号有效。

图 1 | 旧 delay recipe 的长距离恢复失败



来源: `runs/exploratory/delay_parameter_recovery_level0a_v1 · lr=.001, target=5, init_raw=-2`

讲图提示 左上角 delay 只从约 .95 移到 1.14, 远离 target=5; 右上角误差始终远高于 .1 gate。下方显示 loss 在下降、梯度也非零, 所以这不是“完全没有梯度”, 而是有效步长与参数几何不足。

4. Level 0C–0D: 找到能穿过 hard-spike forward 的软 credit

4.1 Level 0C: soft centroid 修复双向长距离方向

在 buffer current 与 subthreshold membrane 两条路径上, production sigmoid + soft centroid + Adam .05 均恢复 15/15, 且 13/13 初始方向正确。这个实验说明连续状态中保留了足够的时间坐标。

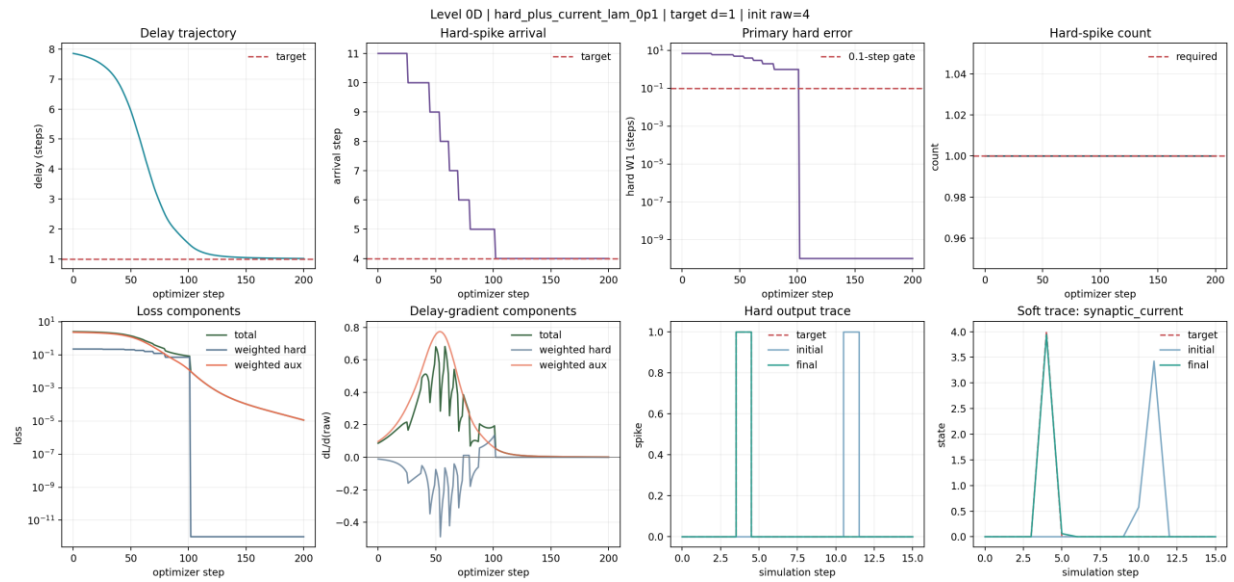
4.2 Level 0D: hard output 保持, 训练 credit 使用 current centroid

最终选择 hard filtered-spike loss + synaptic-current centroid, $\lambda=0.1$ 、Adam .05、200 updates。15/15 恢复且 13/13 方向正确。hard-only 只有 10/15。

- forward endpoint 仍然是硬 LIF spike, 并没有把任务变成纯连续网络。
- 但 delay 的可用方向来自显式的连续 timing auxiliary; 这正是后来 scaffold 依赖的起点。

阶段性判断 到 Level 0D, 我们可以说“系统可被训练到目标时间”, 但不能说“任务标签会自己产生这个 timing signal”。

图 2 | Level 0D 的 hard-forward / soft-credit bridge



来源: `runs/exploratory/delay_hard_output_soft_credit_level0d_v1 · hard + current centroid, lambda=.1`

讲图提示 左上 delay 从约 7.8 回到 target=1；第二幅 hard-spike arrival 以整数阶梯逐步提前；右下 soft current trace 提供连续可微的方向。hard spike 保持一个，但它本身的离散跳变不足以提供稳定双向 credit。

5. Level 1A: 加入 XOR 后, scaffold 成功, task-only 失败

5.1 Stage I 先验证输出接口

固定 d_0 与固定 delay 4 在 $4 \rightarrow 16 \rightarrow 2$ hard-spiking XOR 网络中均通过 5/5 seeds。选出的接口为 $\eta=0$ 、 $lr_w=.01$ 。这证明后续 delay failure 不能归因于 XOR 输出接口本身不工作。

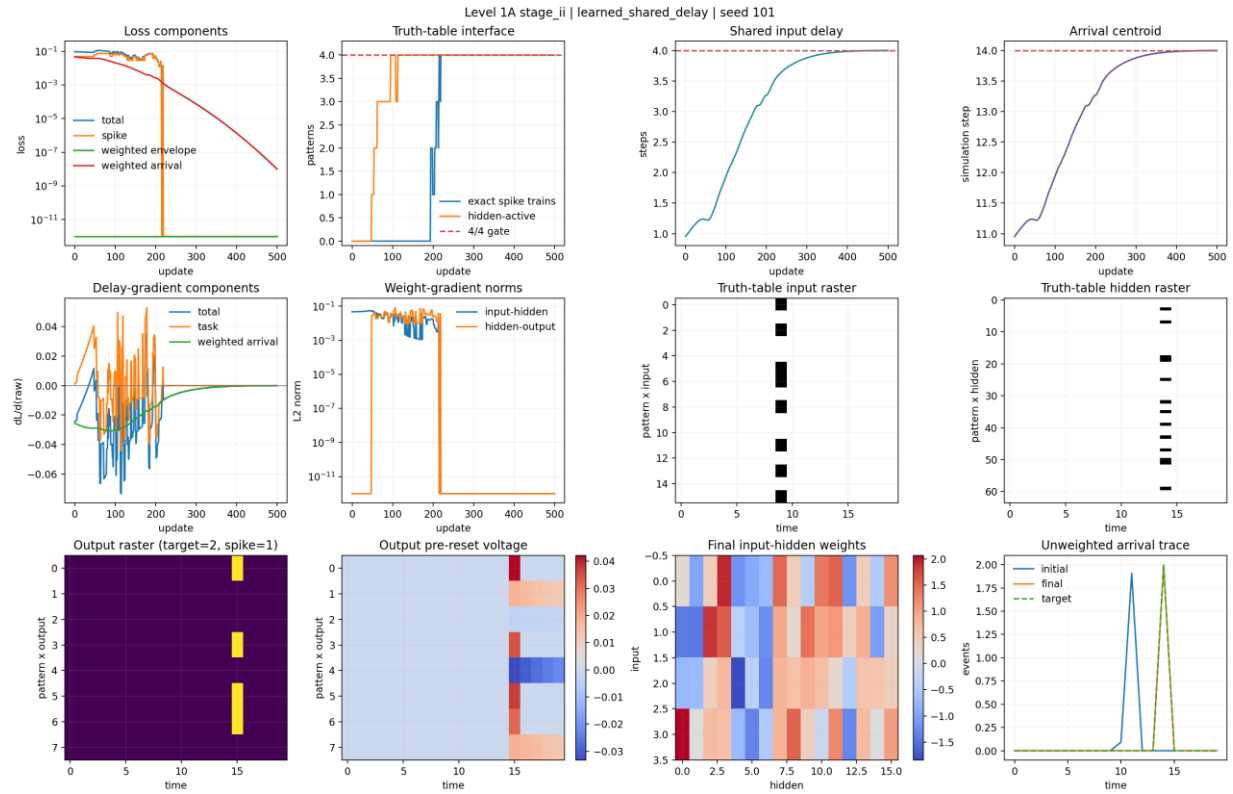
5.2 Stage II 直接比较 task-only 与 timing scaffold

条件	通过率	关键观察
Task-only, $lr_d=.01$	0/10	初始 task 梯度仅 5/10 方向正确
Task-only, $lr_d=.05$	1/10	部分功能成功, 但 schedule/方向 gate 失败
Arrival scaffold, $\lambda=.01$, $lr_d=.01$	10/10	delay 约 4, 最大误差 .002628

最重要的不是 scaffold 的 accuracy, 而是它改变了 delay 梯度方向: 即使 task 与 arrival gradient 在 5/10 cells 中冲突, 总梯度仍能在 10/10 cells 中指向目标。

推荐讲法 Level 1A 是第一次在真实任务里明确看到: 模型会做 XOR, 也能把 delay 调到正确位置, 但需要一个告诉它“应该什么时候到”的老师。

图 3 | Level 1A 的 scaffold-assisted XOR bridge



来源: `runs/exploratory/xor_task_bridge_level1a_v1 · lambda=.01, lr_d=.01, seed 101`

讲图提示 上排第三、四图分别显示 shared delay 接近 4、arrival centroid 接近目标时间 14；truth-table interface 最终 4/4。左中 delay-gradient components 也显示 task 与 arrival 分量会冲突，因此成功不是 task loss 自己发现 timing，而是 auxiliary 修正了方向。

6. Level 1B: 参数维度不是不能训练，但监督强度必须随维度缩放

6.1 原始 Level 1B 的表面失败

同一个 mean scaffold 从 1 个 global delay 扩展到 16 个 per-hidden 或 64 个 per-synapse 坐标时，每个坐标收到的 teacher gradient 被 $1/P$ 稀释。结果 global scaffold 10/10, per-hidden 2/10, per-synapse 0/10。

6.2 Dimension-aware rescue

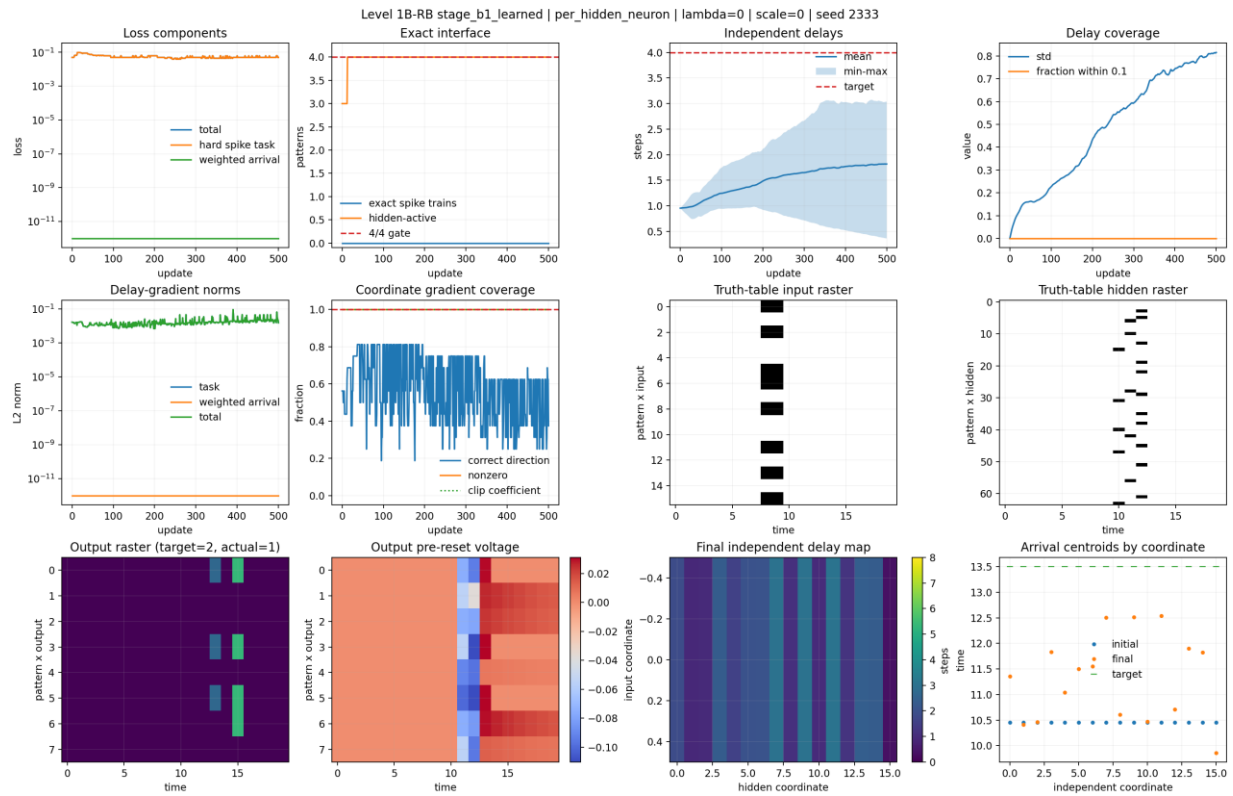
将 lambda 设置为 $.01P$ ，即 global .01、per-hidden .16、per-synapse .64，R1 与 sealed R3 都让三个粒度达到 10/10。由于 per-hidden 参数更少，按预注册规则选 per-hidden。

这说明高维 delay 并非固有不可训练；原始失败很大一部分来自 loss normalization。但这仍是更强的 oracle teacher，不是 task-derived routing。

6.3 Consecutive micro-burst: 最直接的对照

条件	结果	解释
Fixed delay 4	5/5	接口确实需要正确 timing
Global / per-hidden / per-synapse scaffold	各 10/10	显式 schedule 可以稳定实现
Per-hidden task-only	0/10	从头无法发现 homogeneous delay-4 schedule

图 4 | micro-burst 下 task-only 的典型失败



来源: `runs/exploratory/xor_delay_granularity_rescue_microburst_v1 · per-hidden task-only, seed 2333`

讲图提示 Exact interface 曲线最终仍为 0；16 个独立 delay 的均值只到约 1.8，范围不断分散，within-.1 coverage 始终为 0。梯度 norm 非零，但 coordinate correct-direction 只在约 .3-.8 间波动。这里最清楚地展示“有梯度 ≠ 有正确的 schedule credit”。

7. Withdrawal: task loss 能否在 teacher-built basin 内恢复 timing?

7.1 W0/W1 先构造严格的 timing-only damage

五个 per-hidden scaffold foundation 全部通过。随后把全部 16 delays 改成 d3 或 d5：分类仍为 100%，但输出从 target step 15 分别移到 14 和 16。这样把“类别是否正确”和“内部 schedule 是否正确”分离。

7.2 W2: task-only 局部恢复仍然失败

分支	d3	d5	结论
Oracle-delay-only	5/5	5/5	模型与优化器能够恢复
Task-delay-only	exact 3/5; schedule 0/5	exact 0/5; schedule 0/5	主 gate 0/10
Task-joint	exact 5/5; schedule 0/5	exact 3/5; schedule 0/5	weights 可补偿, 但不识别 schedule

7.3 两个内在原因

- 整数边界 credit pathology: right-linear backward 在 $d=5.000$ 选择右侧区间, 平均 task gradient 方向与 $d=4.999$ 相反。
- 不可辨识性: 四个 XOR spike-train constraints 不能唯一决定 16 个 hidden delays; 一旦功能正确, task loss 可变成零, 即使 schedule 远离 oracle。

Gaussian STE $\sigma=1.0$ 修复 aggregate scalar direction, 但 per-hidden P1 仍只有 exact 2/10、全 delay recovery 0/10。这证明整数边界是实因, 但不是唯一原因。

8. K=5 mixed-op: oracle 与 WAD 给出最直接的分叉

8.1 为什么这一组最接近 time multiplexing 问题

五个 query 共用 hidden population，并在五个输出窗口中依次读取。fixed oracle 用 query-conditioned delay 将不同输入送入不同时间槽；WAD 则必须通过训练自己形成这条时间队列。

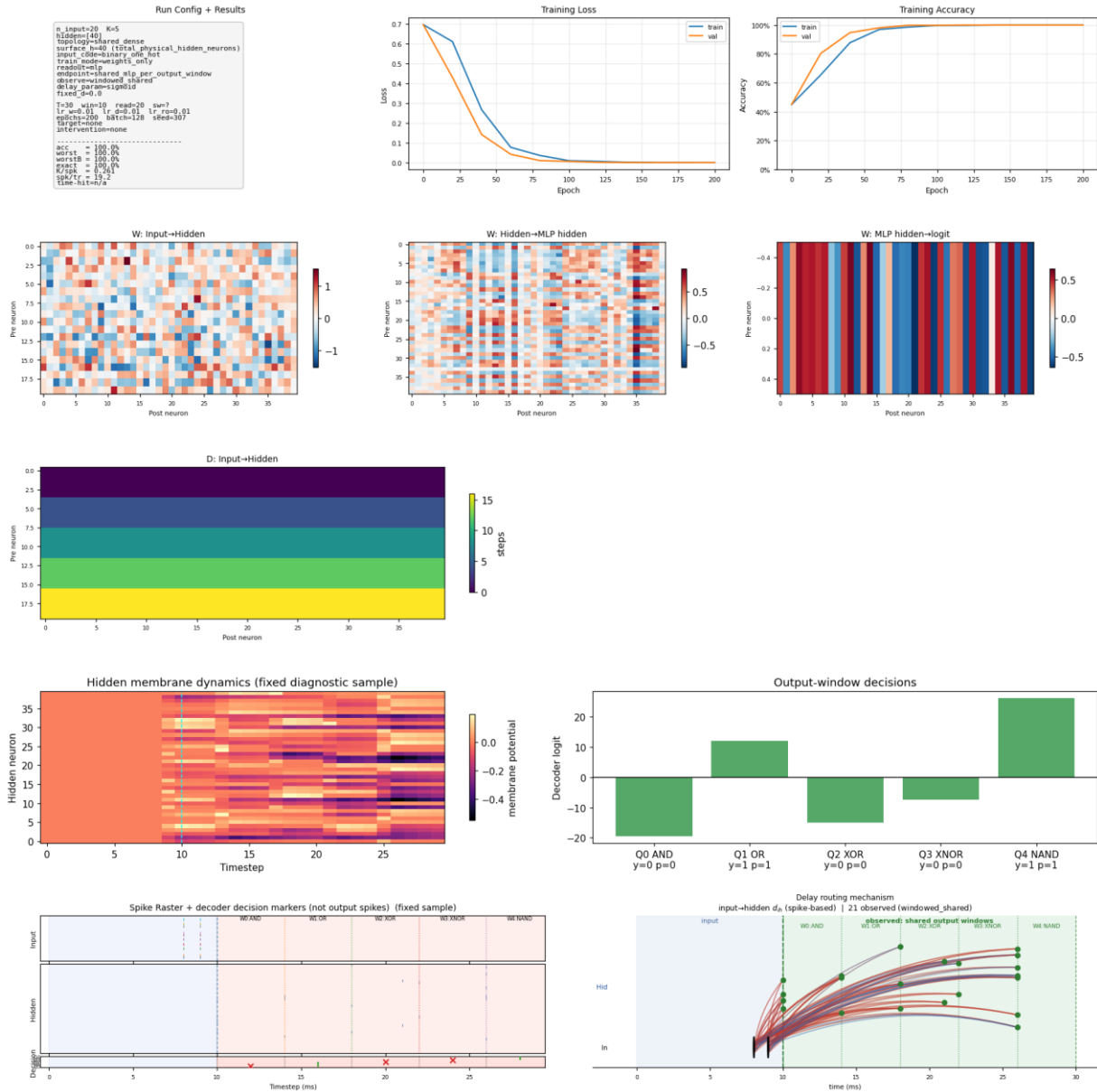
8.2 结果

条件	Worst-query BAcc	窗口活动	解释
Fixed oracle	1.0	几乎 [1,1,1,1,1]	共享网络可以实现五窗口路由
WAD	.5	[1,.967,.0145,0,0]	只覆盖前两窗，后三窗失败
Spatial d0	1.0	独立 spatial path	不是时间复用，但作为功能对照

因此，oracle 是“存在解”的构造性证据，而不是“模型已学会”的证据。WAD delay 聚集在约 .45–.60 个窗口长度，未形成 [0,1,2,3,4] 的 query spacing。

资源限定 在该 K=5 grid 最小点 N=40,T=30，spatial 和 oracle 都已饱和，因此没有 hidden compression 证据；oracle dense MAC/events 为 spatial 的 5 倍，delay buffer 为 17 倍。理论复用不能直接等同于能耗优势。

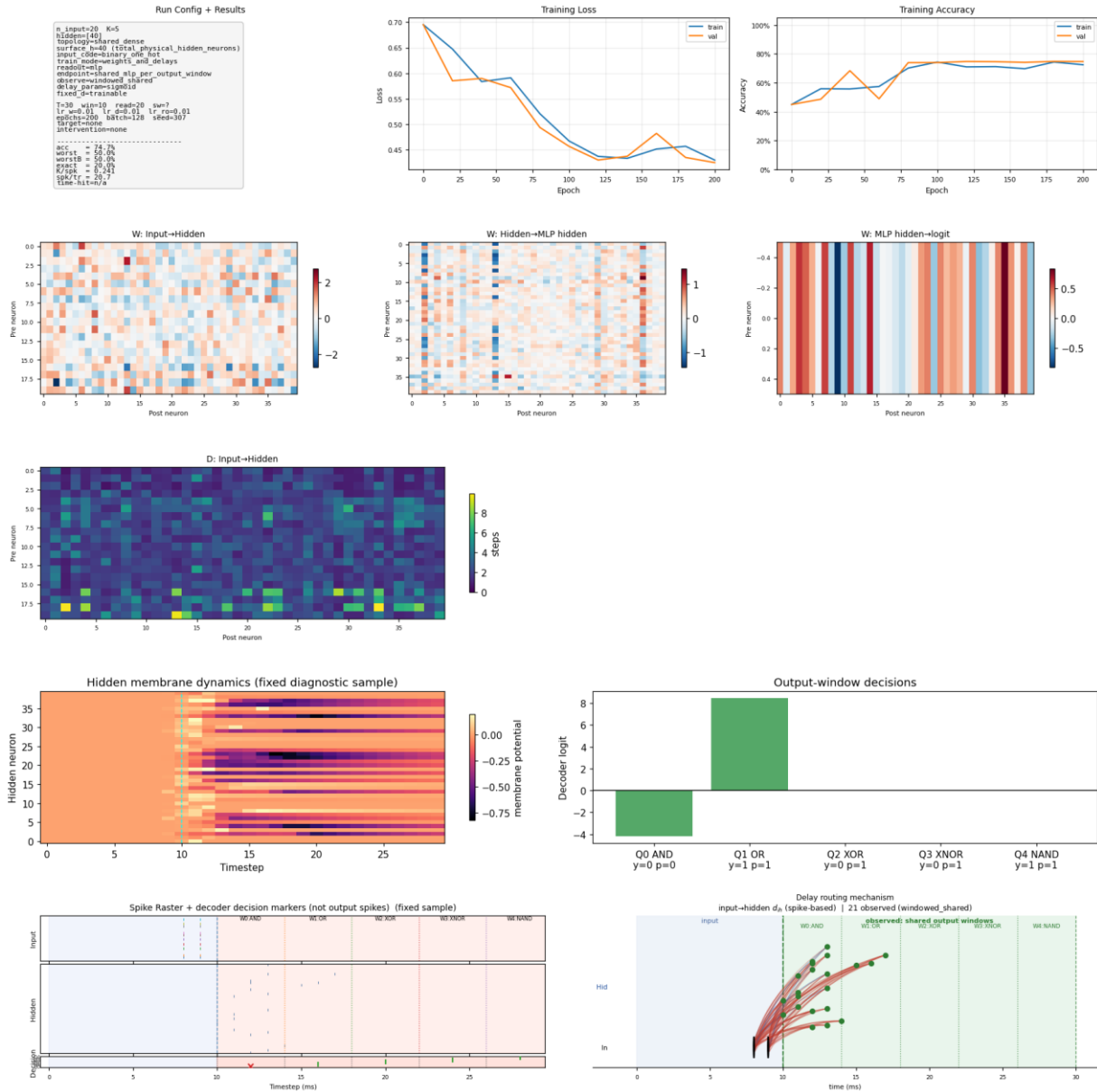
图 5 | K=5 oracle: 时间槽中的共享计算是可行的



来源: runs/exploratory/mixedop_spatial_temporal_surface_preview_v1 · shared temporal oracle, $K=5$, $N=40$, $T=30$

讲图提示 Delay heatmap 呈现五条清晰的 query schedule; 底部 routing mechanism 中, 不同输入通过不同延迟到达五个读取窗口; 五个 output-window logits 均能做出正确决定。它证明 architecture/simulator 存在一个可用的时间复用解。

图 6 | 同一任务下 WAD 没有形成时间队列



来源: runs/exploratory/mixedop_spatial_temporal_surface_preview_v1 · shared temporal WAD, $K=5$, $N=40$, $T=30$

讲图提示 Delay heatmap 仍集中在短延迟；底部事件主要落在最早窗口，后续窗口缺少有效活动。最终 worst-query 为 .5。这是“任务可表达，但当前 WAD 训练没有发现路由”的最直观对照。

9. 从 rate repair 到 V3: 训练目标必须与 schedule endpoint 对齐

9.1 Rate alignment: 先修正输入包与读取窗口

原 rate pilot 的 packet 其实没有对齐输出窗口。修正为 steps 6–9、oracle $d_q=3+q_w$ 后, event8 oracle 在 8 个 candidate $\times$ 3 seeds 全通过; event4 仍因随机性不足以通过 exact-trial。

9.2 Stage B: 五个 query-tied delays 仍学不出来

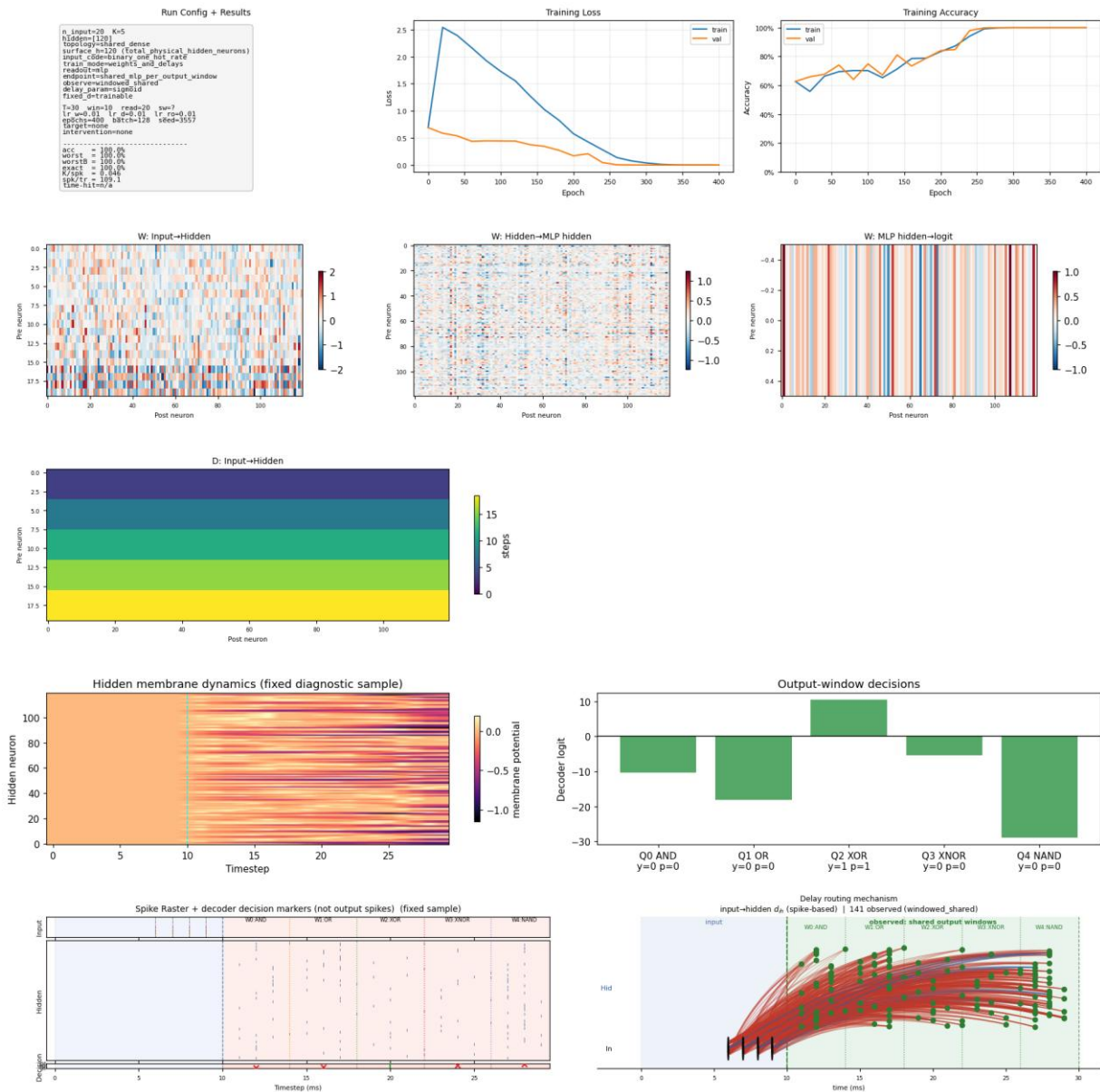
新 seed oracle 3/3; task-only 0/3; routing-assisted .10 也 0/3。梯度很大、route loss 下降, 但 q4 仍无法到达, 说明不是简单的 gradient vanishing。

9.3 V2/V3: 显式 supervision 的目标函数也要匹配 endpoint

- V2 arrival-mass CE: 分类 3/3 完美, 但 q0 被推到 $\text{delay} \approx 1$, $\text{schedule error} \approx 2$ steps, 因此 formal gate 0/3。
- V3 arrival-centroid Huber: 分类与 schedule 联合 gate 3/3, learned delays 约 [3.5, 7.5, 11.5, 15.5, 19.5]。

V3 是一个干净的 supervised routing success: 说明只要提供每个 query 应该进入哪个窗口的明确 teaching signal, 五窗口路由是可训练的。它仍然不是 autonomous WAD。

图 7 | V3 显式 centroid supervision 学出五窗口路由



来源: `runs/exploratory/mixedop_temporal_wad_repair_v3 · arrival-centroid Huber, seed 3557`

讲图提示 Delay heatmap 已形成五条 query-conditioned schedule; 五个窗口均有 hidden activity, classification 接近 100%。这张图应该与前一图 WAD failure 连续展示: 差别不是网络容量, 而是是否提供了与 window center 对齐的 credit。

10. 最新实验：W0/W1 scaffold withdrawal

10.1 W0：显式 scaffold 在新 seeds 上可重复

seeds 3657/3669/3691 的 worst-query 与 exact-trial 均约 1.0，所有窗口活动为 1，最大 schedule error 约 .50 step。W0 通过 3/3。

10.2 W1：突然撤除与渐进撤除出现分叉

W1 arm	3-seed gate	最大 schedule error (3657/3669/3691)	解释
Delay frozen	3/3	.500 / .501 / .500	已学 schedule 可被保存
Centroid continue	3/3	.512 / .510 / .516	持续 teacher 稳定
Abrupt task-only	1/3	.781 / 1.114 / 1.280	功能近完美，但 schedule 漂移
Annealed joint	3/3	.386 / .897 / .501	渐进撤 teacher 可保留

按照 machine decision，withdrawal_retention_passed=true，因为 annealed arm 3/3 通过；但 abrupt task-only 没有稳定通过。W2 尚未运行，配置中 w2_launch=false。

最新结论的准确说法 模型可以在课程学习/退火下维持一个被 teacher 建立的五窗口路由；它仍未证明仅靠 task loss 从头发现路由，也未证明受到扰动后 task-only 能恢复路由。W1 是 warm-start maintenance，不是 autonomous discovery。

注：本页使用 2026-07-17 06:03 生成的 w1_decision.json；截至成稿时尚未看到单独的 W1 narrative results 文档，因此不把它扩大为正式 publication claim。

11. 为什么训练不出来：目前最可信的内在机制

11.1 Credit assignment 不等于 gradient existence

多次失败实验中 delay gradient norm 都非零，甚至 routing loss 明显下降，但梯度在不同 query、不同 delay 坐标和整数两侧的方向并不一致。真正的瓶颈是“梯度是否把每个坐标送到正确的窗口”，而不是“有没有一个非零数”。

11.2 Hard-spike 与离散 delay 造成非光滑、方向不对称的 landscape

输出 spike time 是阶梯函数；归一化 hard-spike centroid 在 spike time 不变时可给零梯度。delay interpolation 在 exact integer 处又需要选择一侧 backward，导致 $d=5.000$ 与 $d=4.999$ 方向翻转。

11.3 Task objective 对内部 schedule 不可辨识

只要输出分类正确，很多不同的 weight-delay 组合都能达到相同 task loss。task-joint 经常恢复功能，却没有恢复 oracle delay；这不是纯优化失败，而是 objective 本身没有要求唯一的内部 route。

11.4 多 query / 多坐标导致 symmetry 与 credit dilution

从同一短 delay 初始化开始，各 query 接收相似的早期 signal，容易一起停在前几个窗口。mean auxiliary 又把每坐标梯度缩小为 $1/P$ ；若没有 query coverage 或 symmetry-breaking，后期 query 很难获得竞争优势。

11.5 长路由与 bounded sigmoid 的几何

靠近 delay support 边界时 sigmoid Jacobian 变小，长距离 q4 movement 更慢。将 target 移到 interior 后有所改善，但 task-only 仍失败，说明它是贡献因素而非唯一根因。

12. 是模型的固有原因，还是我的实现/训练原因？

我的判断 目前不能把失败归为“模型固有不可能”，因为 oracle 和显式 centroid supervision 已经构造出成功解；也不能说只是一个小实现 bug，因为多个被修复的问题之后，task-only 仍持续失败。最合理的结论是：模型可表达，但当前模型-loss-parameterization 组合不具备良好的自主可学习性。

可能原因	当前证据	更像哪一类	如何进一步区分
Simulator / buffer 根本错误	0A、0B current centroid、0C/0D 均成功	基本排除	保留单元测试与 forward identity
旧 LR / budget 太小	0A 明确成立；提高后仍不能修复 task credit	训练 recipe 原因之一	delay-space matched optimizer
整数 backward 选择	W2 找到方向翻转；Gaussian STE 只部分修复	实现/估计器原因之一	连续 timing credit + global anchor
Loss 不识别内部 schedule	task-joint 功能恢复但 schedule 不恢复	任务定义的结构原因	functional causal endpoints, 不强求 oracle 参数
共享 SNN 无复用容量	K=2/K=5 fixed oracle 成功	不支持	更多 seeds、扰动与 matched resource
WAD 自主形成路由困难	多任务、多 encoding、多 seed 一致失败	当前方法的实质限制	query-aware prior、coverage loss、curriculum withdrawal
所有 delayed SNN 都学不会	证据不足，且显式 supervision 可学	不能下结论	外部 temporal benchmark 与替代模型

对“是不是我的原因”的简短回答

有一部分确实来自实现与实验设计：旧 LR 太小、mean loss 稀释、rate packet 未对齐、整数 backward 不对称。这些已经通过分层实验被逐个识别并部分修复。

但修复之后 task-only 仍然失败，而且 oracle / teacher 一直成功。因此剩余问题不是“粗心写错一行代码”这么简单，而是当前 SNN 的 hard temporal dynamics 与 task loss 之间缺少一个稳定、可辨识的时间信用通道。

13. 汇报收尾讲稿

建议原话 这组实验给我的最终认识是：asynchronous delayed SNN 的共享 hidden population 确实存在时间复用解，oracle 和显式 centroid routing 已经把这个解构造出来了。但当前网络不会仅靠任务标签稳定地自己发现这个解。我们修复了学习率、loss normalization、输入对齐和整数边界 backward 等问题，仍然看到 task-only 的方向冲突、schedule 不可辨识和后期窗口 coverage collapse。所以现在最诚实的结论是“capacity exists, autonomous trainability is not established”。

可以回答的三个追问

4. 这算不算 time multiplexing？架构可表达性层面算 temporal reuse feasibility；学习方法层面尚不能称 autonomous time multiplexing。
5. 为什么 oracle 重要？它是构造性存在证明：排除网络容量、simulator 和 readout 完全不可能；但它不能证明训练能找到解。
6. 下一步最关键是什么？完成 W2 local restoration；然后用 target-free、query-aware 的结构约束或连续 output-time credit，测试是否能在不直接给 oracle schedule 的情况下恢复功能性路由。

建议下一步优先级

- 先完成 K=5 W2：对 uniform -2 与阶梯 perturbation 比较 frozen、centroid、task-only restoration。
- 把 headline endpoint 从“delay 等于 oracle 参数”改为功能性因果路由：窗口活动、query-specific intervention、delay shuffle、schedule perturbation 后的预测变化。
- 设计 target-free 的 symmetry breaking：query-aware low-rank/affine schedule、coverage/ordering constraint 或 curriculum；必须与 fixed oracle、d0、non-learned schedule 对照。
- 若仍失败，论文应转向 negative methodology：非零 gradient、平均 accuracy 与漂亮 raster 都不能证明 learned temporal routing。

附录 A | Level 0 到最新实验的完整索引

实验	规模/状态	一句话结论
delay_parameter_recovery_level0a_v1	75/75	sigmoid delay 可恢复; 旧 .001/200 recipe 不足
delay_temporal_credit_level0b_v1	180/180, fail	buffer 可导, 但 hard/filtered credit 不可靠
delay_soft_trace_credit_level0c_v1	360/360, pass	soft centroid 提供双向连续 timing credit
delay_hard_output_soft_credit_level0d_v1	135/135, pass	hard output + current centroid bridge 通过
xor_task_bridge_level1a_v1	90+85 cells	XOR 接口稳; task-only 失败, scaffold 10/10
xor_delay_granularity_level1b_v1	60 cells + controls	原 mean scaffold 不随维度扩展
xor_delay_granularity_rescue_level1br_v1	50+30 cells	$\lambda \propto P$ 修复 per-hidden/per-synapse
xor_delay_granularity_rescue_microburst_v1	5+40 cells	三种 scaffold 通过; task-only 0/10
xor_task_derived_timing_withdrawal_v1	5+10+45 cells	retention 有, task-only local restoration 0/10
xor_integer_boundary_credit_preflight_v1	P0 100 probes + P1 10	Gaussian STE 修复平均方向, 但 recovery 仍失败
mixedop_spatial_temporal_surface_preview_v1	K5 36 cells	oracle 1.0, WAD .5; K8 暂停
mixedop_rate_wad_surface_calibration_v1	10 pilot cells	rate 增事件但未救 WAD; oracle 原设计未对齐
mixedop_rate_alignment_repair_v1	48+9 cells	对齐后 oracle 成功, task-only/assisted 仍 0/3
mixedop_temporal_wad_repair_v2/v3	12+9 cells	V3 centroid-supervised schedule 3/3
mixedop_temporal_wad_scaffold_withdrawal_v1	W0 3 + W1 12	W0 pass; W1 annealed 3/3, abrupt 1/3; W2 未运行

附录 B | 主要权威来源

- docs/CLAIMS_LEDGER.md
- docs/RESULTS_DELAY_PARAMETER_RECOVERY_LEVEL0A_V1.md
- docs/RESULTS_DELAY_TEMPORAL_CREDIT_LEVEL0B_V1.md
- docs/RESULTS_DELAY_SOFT_TRACE_CREDIT_LEVEL0C_V1.md
- docs/RESULTS_DELAY_HARD_OUTPUT_SOFT_CREDIT_LEVEL0D_V1.md
- docs/RESULTS_XOR_TASK_BRIDGE_LEVEL1A_STAGE_II.md
- docs/RESULTS_XOR_DELAY_GRANULARITY_RESCUE_MICROBURST_STAGE_B1.md
- docs/RESULTS_XOR_TASK_DERIVED_TIMING_WITHDRAWAL_W2.md
- docs/RESULTS_XOR_INTEGER_BOUNDARY_CREDIT_PREFLIGHT_V1.md

- docs/RESULTS_MIXEDOP_SPATIAL_TEMPORAL_SURFACE_PREVIEW_V1_K5.md
- docs/RESULTS_MIXEDOP_RATE_ALIGNMENT_REPAIR_STAGE_B.md
- docs/RESULTS_MIXEDOP_TEMPORAL_WAD_REPAIR_V3.md
- docs/RESULTS_MIXEDOP_TEMPORAL_WAD_SCAFFOLD_WITHDRAWAL_W0.md
- docs/generated/mixedop_temporal_wad_scaffold_withdrawal_v1/w1_decision.json

所有插图均直接取自 runs/exploratory 下对应正式 cell 的 runtime diagnostic_panel.png；未使用 smoke cell，也未对图像做裁剪或内容修改。